

s1.07 — Logging service / log-file storage layer

Index

- [Interview framing](#)
- [Requirements — IC3 baseline](#)
- [Capacity and consequence — IC3 baseline](#)
- [Architecture and flow — IC3 baseline](#)
- [API and data model — IC3 baseline](#)
- [Group commit, rotation, and crash recovery — IC3 baseline](#)
- [Time-range query and retention — IC3 baseline](#)
- [Failure, operations, and security](#)
- [Alternatives and IC4 stretch](#)
- [Level signal — IC3 baseline vs IC4 stretch](#)
- [Interview pushback and validation](#)
- [Recall card](#)
- [References](#)

Interview framing

- Target: OCI IC3 ownership of a bounded platform component.
 - `IC3 baseline` is what must land in every phase.
 - `IC4 stretch` adds architecture changes under stronger constraints.
 - These are preparation labels, not Oracle's official rubric.
- Evidence: retain the first-hand report attribution in [the bank](#).
 - The architecture below is a proposed interview answer.
 - It does not claim to reproduce Oracle's implementation or an interviewer's accepted answer.
- Why this is hard
 - Naive: every producer appends to one file under a lock and calls `flush()` after each line.
 - First break: at `10,000 records/s` the lock and per-record flush serialize producers, and `flush()` still does not mean the bytes reached disk.
 - Second break: a crash leaves a torn frame, a query reads a half-written record, and retention deletes a file a reader is still scanning.
 - Minimum primitives
 - A byte-bounded queue
 - One writer per active segment
 - Checksummed frames
 - A published durable prefix
 - A manifest that readers pin.
- 🗨️ "Acceptance, local durability, and remote durability are three different promises; the design is the state machine that upgrades a record from one to the next."
- Opening script
 - "I will treat this as a host-local segmented log with a central upload path, not a search engine."
 - "First I define the acknowledgment modes, because 'written' means something different for a debug line and an audit record."
 - "Then I size the queue, segment, and flush batch from the record rate and the disk budget."
 - "The architecture is producers into a bounded queue, one writer per active segment, and a manifest that readers and the uploader consume."
 - "The deep dive is group commit, rotation, and what crash recovery can and cannot trust."

- “I will close with retention versus readers, failure modes, and where I would change the design for an audit contract.”

Phase	Clock	Deliverable
Intro and behavioral slice	00:00–05:00	90-second pitch; confirm the prompt
Requirements	05:00–11:00	Scope, acknowledgment modes, primary invariant
Estimates and contract	11:00–17:00	Segment and batch sizing; API semantics
Architecture	17:00–30:00	Diagram, data ownership, one append trace
Deep dive	30:00–45:00	Group commit, rotation, crash recovery, range query
Operations and tradeoffs	45:00–50:00	Failure table, retention, tradeoff, wrap script
Wrap-up	50:00–54:00	Four-sentence recap and weakest assumption
Questions for interviewer	54:00–60:00	Two role-specific questions

- Clock note
 - If the interviewer states 45 minutes, drop the intro to 2 minutes and compress each phase proportionally.
 - The interviewer's clock wins over this table.

Requirements — IC3 baseline

- Numerical targets are interview assumptions unless an adjacent primary source states otherwise.

Question	Assumed answer	Why it changes the design
Library, host daemon, or central service?	Host daemon that owns local files and uploads sealed segments	One writer per host; local disk is the first durability boundary
What does “written” mean to the caller?	Three modes: best-effort, local-durable, replicated	Decides what the API returns and what a loss window can claim
Query shape?	Time-range scan by ingest time; full-text search is a separate derived index	A sparse offset index suffices; no inverted index on the hot path
Ordering key?	Writer-assigned ingest sequence; event time is a carried field	Late event time cannot break append order
Retention?	24 h local within 200 GB; longer retention lives centrally	Disk budget forces the upload and deletion policy
Multi-tenant streams?	Yes; streams are tenant-scoped with per-stream quotas	Queue bytes, disk share, and authorization are per stream
Record size bound?	200 B typical, 64 KB maximum	Fixes frame layout and the memory bound per batch

- Functional scope
 - Accept structured records into a named stream with a chosen acknowledgment mode.
 - Append records to immutable, checksummed, size-bounded local segments.
 - Seal and publish segments to a manifest that readers and the uploader consume.
 - Serve bounded time-range queries over local segments.
 - Upload sealed segments to object storage and reclaim local disk under retention.
- Acknowledgment modes
 - `BEST_EFFORT`: accepted into bounded memory
 - May be lost on process crash.
 - `LOCAL_DURABLE`: acknowledged after the required local file flush succeeds.
 - `REPLICATED`: acknowledged after the chosen remote durability boundary.
 - A local durable append does not survive loss of the only disk or host.

Requirement	Target	Enforced by
Ingest rate per host	10,000 records/s at 200 B/record	Byte-bounded queue with explicit admission result

Requirement	Target	Enforced by
Best-effort enqueue latency	p99 < 1 ms under admitted load	Reserve-and-copy on the producer thread; no I/O on that thread
Group flush trigger	50 ms or 1 MB, whichever fires first	Writer timer plus byte trigger
Local-durable acknowledgment	p99 < 100 ms under healthy storage	One flush per batch; futures complete after the durable prefix advances
Query latency	p99 < 1 s for a bounded five-minute range	Sparse index plus pinned manifest; bounded page size
Local retention	At most 24 h within a 200 GB budget	Retention worker deletes oldest unpinned sealed segments first
Central freshness	Sealed segment uploaded within 5 min under healthy network	Uploader with retry; oldest-unshipped-segment alarm
Maximum record size	64 KB	Rejected at admission before queue reservation
Data-plane resilience	Append survives a 1 h registry outage for preauthorized streams	Versioned cached configuration with a 1 h policy validity budget

- Invariants
 - One writer owns each active segment's byte offset.
 - Published durable offset never exceeds a successfully flushed prefix.
 - A record is either fully framed and checksummed or rejected during recovery.
 - Retention never deletes a segment still pinned by an active reader.
 - A claimed loss window includes queue residence and disk stalls.
- Non-goals
 - Full-text or field search
 - That is a derived index over uploaded segments.
 - Global ordering across hosts
 - Ordering is per stream per writer.
 - Exactly-once delivery to downstream consumers.
 - A tamper-evident audit ledger
 - That is a stricter stream class with its own retention and permissions.

Capacity and consequence — IC3 baseline

- Raw write volume
 - $10,000 \times 200 \text{ B} = 2 \text{ MB/s}$.
 - Daily input is 172.8 GB before framing/index overhead.
 - Seven days would require about 1.21 TB before compression.
 - A 200 GB local budget holds about 27 h of raw input, so it cannot promise seven-day local retention at this load.
 - Decision: cap local retention at 24 h
 - Anything longer lives in object storage, so upload is part of the durability story.
- Segment size
 - A 64 MB segment fills in approximately 32 s.
 - This bounds recovery scan and upload unit size.
 - Compression is performed on sealed segments and consumes CPU/I/O.
 - Do not assume a fixed compression ratio.
 - Decision: 64 MB keeps the worst-case crash scan to one segment and gives the uploader a unit large enough to amortize per-object overhead.
- Flush batch
 - At 2 MB/s a 50 ms timer yields batches of about 100 KB, roughly 500 records.

- That is about 20 flushes/s regardless of record rate.
- Per-record flushing would demand 10,000 flushes/s, an order of magnitude beyond what a typical single disk sustains for synchronous flushes.
- Decision: group commit is mandatory for durable mode
 - The flush count, not the record count, is the storage budget.
- Buffering
 - A 64 MB queue holds about 32 s of raw records at the example rate.
 - With unlimited writer delay, a 50 ms flush timer does not bound data age to 50 ms.
 - Decision: track oldest queued record age and distinguish the timer target from a hard durability SLA
 - A disk stall turns the whole queue into the loss window.
- Sparse index
 - One entry per 64 KB block gives 1,024 entries per 64 MB segment, about 16 KB at 16 B per entry.
 - 200 GB / 64 MB ≈ 3,200 local segments imply about 51 MB of index.
 - Decision: the index fits in memory, so a range query costs one positioned read per candidate segment rather than a scan from the segment start.

Architecture and flow — IC3 baseline

```

Stream registry (control plane) --cached config--> host daemon
                                     |
Application producers -> byte-bounded queue -> single segment writer
                                     |
                                     active segment + sparse index
                                     |
                                     seal / manifest publication
                                     |
sealed files -> uploader -> object store

Range query -> pinned segment manifest -> framed record scans
Central search index <- durable segment/object references

```

- Stream registry (control plane) owns stream definitions, retention, and authorization.
 - Mechanism: the daemon polls it and caches last-known-good config.
 - Cost: a stale cache applies old quota or retention for one poll interval
 - Static stability is bought with bounded staleness.
- Producer path owns admission.
 - Mechanism: reserves queue bytes before copying the record
 - Returns accepted, rejected, or a durability future by mode.
 - Cost: rejection is a real outcome the caller must handle
 - Best-effort loss is explicit, not silent.
- Queue owns the memory bound.
 - Mechanism: byte-counted batches with a per-stream share
 - Oldest-record age is a first-class metric.
 - Cost: overload drops or blocks
 - The queue cannot be both lossless and bounded.
- Writer owns the active segment and the durable prefix.
 - Mechanism: drains by size or oldest-record deadline, handles partial writes, flushes before advancing the durable offset, serializes rotation with append.
 - Cost: throughput is capped by one encode-and-write loop
 - Scaling needs more partitions, not more writers per segment.
- Sealer and uploader own publication and central durability.
 - Mechanism: seal on size or age, write a checksummed footer, publish the manifest generation, upload under a stable object identity.

- Cost: two durability states, local and central, and the operator must observe both.
 - Query owns visibility.
 - Mechanism: pins a manifest generation and its segments, reads valid frames up to the visibility boundary, releases pins.
 - Cost: pins delay retention
 - A leaked pin is a disk leak.
- Reserve bounded queue space and assign a record identity.
- Enqueue the record or return an explicit admission failure.
- Writer appends a length-prefixed checksummed frame.
- Writer flushes the batch when the byte/time trigger fires.
- Advance the durable prefix only after successful flush.
- Complete durable acknowledgment futures for records in that prefix.
- Seal and publish segments for range queries and asynchronous upload.

API and data model — IC3 baseline

Surface	Contract	Conflict/error behavior
<code>append(stream, record, mode)</code>	Result or future with record sequence	Best-effort acceptance is distinct from durable acknowledgment; oversize or over-quota returns an explicit rejection
<code>appendBatch(stream, producerEpoch, firstSeq, records)</code>	Idempotent batch keyed by producer epoch and sequence	Replay of an acknowledged <code>(epoch, seq)</code> returns the prior result; a stale epoch is rejected
<code>flush(stream, throughSequence)</code>	Wait for a required durable prefix	Deadline/error is visible to the caller; never reports success on a failed flush
<code>query(stream, from, to, cursor, limit)</code>	Bounded page with an ordering definition	Cursor pins segment/offset or snapshot generation; an expired pin returns a documented expiry, not silent gaps
<code>close()</code>	Stop admission, drain, flush, seal	Bounded shutdown; reports the unflushed remainder

State	Key	Fields/invariant
Record frame	<code>(stream, writer_epoch, sequence)</code>	Length, checksum, ingest time, event time, severity, payload
Producer checkpoint	<code>(stream, producer_id)</code>	Epoch, last acknowledged sequence; makes batch replay idempotent
Segment	<code>(stream, writer_epoch, segment_sequence)</code>	Path, start/end offset, actual time range, state, checksum
Manifest	<code>(stream, generation)</code>	Immutable ordered segment references and durable boundary
Upload progress	<code>(segment_id, destination)</code>	Stable object identity, verified checksum, committed state

- Reference DDL
 - Keys and uniqueness match the state table above and the [companion schema](#).
 - Record frames, segment bytes, and the manifest file live on local disk; only the stream registry, segment index, producer checkpoints, manifest generations, and upload progress are relational.

```

CREATE TABLE log_streams (
    -- stream registry (control plane)
    stream_id text NOT NULL, tenant_id text NOT NULL, name text NOT NULL,
    stream_class text NOT NULL CHECK (stream_class IN ('DEBUG','AUDIT')), -- audit has stricter retention
    and permissions
    retention_seconds bigint NOT NULL CHECK (retention_seconds > 0),
    quota_bytes bigint NOT NULL CHECK (quota_bytes > 0),
    upload_required boolean NOT NULL, -- local deletion waits for a verified central upload
    version bigint NOT NULL,
    PRIMARY KEY (stream_id),
    UNIQUE (tenant_id, name)
);
CREATE TABLE producer_checkpoints (
    stream_id text NOT NULL, producer_id text NOT NULL,
    epoch bigint NOT NULL CHECK (epoch >= 0),
    last_acked_seq bigint NOT NULL CHECK (last_acked_seq >= 0),
    last_offset bigint NOT NULL CHECK (last_offset >= 0),
    updated_at timestamptz NOT NULL,
    PRIMARY KEY (stream_id, producer_id),
    FOREIGN KEY (stream_id) REFERENCES log_streams
);
CREATE TABLE segments (
    -- segment index; frames stay in the segment file
    stream_id text NOT NULL, writer_epoch bigint NOT NULL, segment_sequence bigint NOT NULL,
    path text NOT NULL,
    start_offset bigint NOT NULL CHECK (start_offset >= 0), end_offset bigint,
    ingest_min timestamptz NOT NULL, ingest_max timestamptz,
    event_min timestamptz, event_max timestamptz, -- true observed range; may overlap neighbours
    state text NOT NULL CHECK (state IN ('ACTIVE','SEALED','UPLOADED','DELETED')),
    size_bytes bigint NOT NULL CHECK (size_bytes >= 0),
    footer_checksum text, -- written at seal; null only while ACTIVE
    PRIMARY KEY (stream_id, writer_epoch, segment_sequence),
    UNIQUE (path),
    FOREIGN KEY (stream_id) REFERENCES log_streams,
    CHECK (end_offset IS NULL OR end_offset >= start_offset),
    CHECK (state = 'ACTIVE' OR (end_offset IS NOT NULL AND footer_checksum IS NOT NULL))
);
CREATE UNIQUE INDEX segments_one_active ON segments (stream_id) WHERE state = 'ACTIVE';
CREATE INDEX segments_time_range ON segments (stream_id, ingest_min);
CREATE INDEX segments_retention ON segments (stream_id, ingest_max)
    WHERE state IN ('SEALED','UPLOADED');
CREATE TABLE manifests (
    stream_id text NOT NULL, generation bigint NOT NULL CHECK (generation >= 0),
    path text NOT NULL, -- fsync'd manifest file; holds the ordered segment references
    durable_offset bigint NOT NULL CHECK (durable_offset >= 0),
    published_at timestamptz NOT NULL,
    PRIMARY KEY (stream_id, generation),
    FOREIGN KEY (stream_id) REFERENCES log_streams
);
CREATE TABLE upload_progress (
    stream_id text NOT NULL, writer_epoch bigint NOT NULL, segment_sequence bigint NOT NULL,
    destination text NOT NULL,
    object_ref text NOT NULL, -- stable object identity so retries overwrite, never duplicate
    verified_checksum text,
    state text NOT NULL CHECK (state IN ('PENDING','UPLOADING','COMMITTED','FAILED')),
    attempts integer NOT NULL DEFAULT 0 CHECK (attempts >= 0),
    updated_at timestamptz NOT NULL,
    PRIMARY KEY (stream_id, writer_epoch, segment_sequence, destination),
    FOREIGN KEY (stream_id, writer_epoch, segment_sequence) REFERENCES segments
);
CREATE INDEX upload_progress_pending ON upload_progress (updated_at) WHERE state <> 'COMMITTED';

```

- Not captured by the DDL
 - The record frame (length | crc32c | writer_epoch | sequence | ...), the sparse offset index, durable_offset on the writer thread, and reader pins with their leases are in-memory or on-disk structures, not rows.
 - The manifest row records the published generation and durable boundary; the ordered segment list itself is the fsync'd file.

- Deviation from the companion: keys are `stream_id`-scoped as in the state table, with `tenant_id` a column of the registry only; the companion's `partition_id/base_offset` segment key becomes `(writer_epoch, segment_sequence)`; the state table's `segment_id` is that composite key.
- Separate time concepts.
 - Ingest sequence defines append order even if the wall clock moves backward.
 - Event time can arrive late or out of order.
 - Preserve actual event-time minimum/maximum if queries use event time.
 - Never replace a real minimum with `max(seen)` and call the range correct.
- Frame convention
 - `length | crc32c | writer_epoch | sequence | ingest_time | event_time | severity | payload`.
 - The length prefix bounds the read
 - The checksum covers everything after it.
 - A format version in the segment header lets an old reader refuse rather than misparse.

Group commit, rotation, and crash recovery — IC3 baseline

```
WRITE_BATCH(batch):
    encode bounded frames
    loop until every frame byte is written or an error occurs
    flush file data according to the selected durability mode
    if flush succeeds:
        advance durable_offset
        acknowledge records through durable_offset
    otherwise:
        leave durable_offset unchanged
        fail/block pending durable acknowledgments by contract
```

- Concurrency boundary
 - Producers touch only the queue
 - The writer thread solely owns the file descriptor and `durable_offset`.
 - Advance `durable_offset` before completing acknowledgment futures.
 - Dispatch completions on a bounded executor so inline user callbacks cannot block the writer thread.
- Rotation sequence
 - Stop appending to the old segment under writer ownership.
 - Write the old segment's final footer and checksum.
 - Flush the complete sealed segment before publishing it.
 - Create the next segment with a unique writer epoch and sequence.
 - Persist the new segment header.
 - Write and `fsync` the temporary manifest before atomic replacement where supported.
 - Apply required directory synchronization for crash-durable namespace updates.
 - Publish the manifest generation to readers and upload workers.
- Filesystem boundary
 - `write` or language-level buffered flush alone does not establish disk durability.
 - On Linux, file `fsync` does not necessarily persist the containing directory entry.
 - `Fdatasync` must persist metadata needed to retrieve the data, including file size when required.
 - It can omit unrelated metadata such as access timestamps.
 - Neither file-level call alone establishes directory-entry durability.
 - Exact persistence behavior must match the target filesystem and storage stack.
 - See [fsync semantics](#).
- Recovery
 - Read the last durable manifest.
 - Scan the final active segment using record lengths and checksums.

- Truncate only an incomplete/torn tail under the recovery policy.
- Treat corruption in the middle of an allegedly durable segment as corruption, not an ordinary tail truncation.
- Reconcile orphan complete segments from their headers before deleting them.
- Exact failure: a crash after bytes reach the page cache but before `fsync` can leave a valid frame that was never acknowledged.
 - Retaining a valid unacknowledged frame is allowed by the append contract.
 - Exact replay deduplication requires a stable producer event ID, not only the writer's new epoch and sequence.
- Writer identity
 - Use a new epoch on restart to avoid sequence reuse.
 - A daemon or OS-enforced ownership lock prevents two writers sharing one active file.
 - Distributed ownership additionally needs fencing at the storage boundary.

Time-range query and retention — IC3 baseline

- Ingest-time query baseline
 - Use ingest sequence for append order.
 - A wall-clock ingest timestamp can move backward and cannot be assumed sorted without a clamping contract.
 - For wall-clock ranges, use interval-aware segment summaries or a declared logical ingest clock.
 - Find the first relevant indexed block, then scan framed records.
 - Include active data only up to the reader's pinned visibility boundary.
 - Concrete cost: a five-minute range at 2 MB/s is about 600 MB across roughly ten segments
 - The page limit bounds bytes per call.
- Event-time query follow-up
 - Late events can make segment event-time ranges overlap.
 - Binary-searching only `t_min` and starting at one predecessor can skip older overlapping segments.
 - Use interval-aware candidate selection or a bounded scan of all overlapping range summaries.
 - Return sorted event-time results only if the merge cost is within the query budget.
- Compression
 - Whole-file compression may require decompressing from the beginning.
 - Block compression with an offset index preserves bounded seeking.
- Retention
 - Delete oldest eligible sealed segments after reader pins expire.
 - Define whether local deletion requires a verified central upload.
 - If central upload is mandatory and unavailable, backpressure or reject according to stream mode.
 - A retention requirement cannot be satisfied by silently deleting unshipped mandatory records.
 - Exact failure: a pin with no expiry plus a stuck uploader fills the disk
 - Pins carry a lease, and disk runway below one hour pages before admission fails.

Failure, operations, and security

Failure	Blast radius	Detection	Degraded behavior	Recovery
Process dies before flush	Unflushed records on one host	Restart recovery	Best-effort loss possible; durable futures never completed	Scan valid tail; resume with new epoch
Disk full	All streams on the host	Free bytes and writer error	Drop/count best-effort logs; fail durable admission	Restore space under retention policy

Failure	Blast radius	Detection	Degraded behavior	Recovery
Disk flush stalls	Pending durable appends	Oldest queue age and flush latency	Bound waiting producers	Diagnose storage; fail over only with known durable state
Uploader or object store unavailable	Central freshness	Oldest unshipped segment	Continue within local budget; mandatory streams backpressure	Retry stable object IDs; verify checksums
Stream registry (control plane) unavailable	New streams and policy changes	Registry poll failures	Data plane appends on cached config	Refresh config; reconcile quota and retention drift
Query races with deletion	Pinned segments	Reader reference count	Retention waits or query returns documented expiry	Snapshot/pin lifecycle
Record framing corrupt	Segment tail or middle	Length/checksum validation	Quarantine invalid durable segment	Restore replica or escalate data-integrity incident
Wall clock jumps backward	Time-range results on one host	Ingest time below previous frame	Sequence still orders append	Record skew in header; queries key on sequence
Two writers after a failed restart	One stream's active segment	Ownership lock or epoch mismatch	Refuse the second writer	Fence by epoch; recover under surviving owner

- Observe
 - Accepted versus durable acknowledgment counts.
 - Queue bytes and oldest age.
 - Flush latency.
 - Dropped records by mode and severity.
 - Disk runway in hours at current rate.
 - Unshipped segment age.
- Retry discipline
 - Uploader retries with backoff and jitter against a stable object identity, so a retry cannot create a second object.
 - Producers retry only admission timeouts and replay with the same (epoch, seq) so the checkpoint dedups.
 - A flush error is not retried in place
 - The writer surfaces it and recovery owns the segment.
- Tenant isolation
 - Streams are tenant-scoped
 - Authorization is checked at append and query, not only at creation.
 - Per-stream queue-byte and disk-share quotas stop a noisy tenant from evicting another's durable records.
 - Overload sheds best-effort records by severity inside the offending stream only.
 - Redact secrets before storage
 - Bound record sizes to prevent memory abuse.
 - Retention workers hold least-privilege deletion rights
 - Audit retention is separate from debug-log retention.
- Rollout and rollback safety
 - The segment header carries a format version
 - A new daemon reads old segments and an old daemon refuses new ones instead of misparsing.
 - Manifest schema changes follow expand, migrate, contract, so rollback never leaves the manifest unreadable.
 - Deploy drains the queue under a bounded SIGTERM deadline and reports the unflushed remainder.
 - Canary on a host subset
 - Watch dropped-record and flush-latency deltas.

Alternatives and IC4 stretch

Option	What it is	Strength	Weakness	Choose when
Per-record durable append	Flush each required append	Simple acknowledgment boundary	More flush overhead	Low-volume strict local durability
Group commit	Flush batches	Amortizes storage cost	Pending records wait and can fail	Baseline durable throughput
Memory-only enqueue	Async best-effort logger	Low producer latency	Crash loss and overload drops	Noncritical diagnostics
Direct central ingestion	Network append	Central visibility	Network dependency on hot path	Remote durability is explicitly required
Local segments plus upload	Local first, remote later	Tolerates bounded central outages	Two durability/freshness states	Host diagnostics with central search

- IC4 stretch — audit contract
 - If an operation requires durable audit before success, best-effort dropping is invalid.
 - Commit the business operation and durable audit intent in one transaction when possible.
 - Ship through an outbox and expose reconciliation gaps.
 - Local logging alone is not an atomic business/audit transaction.
- IC4 stretch — 10× load
 - At 1,000 hosts, raw volume is about 172.8 TB/day.
 - Central indexing, retention, and compression strategy dominate cost.
 - Index only necessary fields and keep immutable raw objects for the agreed retention.
 - On one host, 100,000 records/s exceeds one writer thread's encode loop
 - Partition the stream so each partition keeps its own single writer.
- IC4 stretch — regional durability
 - Select whether acknowledgment requires a remote copy.
 - Remote acknowledgment increases latency and availability coupling.
 - State RPO/RTO for the specified host or regional failure.
- IC4 stretch — cost and blast radius
 - Object storage bytes dominate cost at fleet scale
 - Tier sealed segments by age and index only the hot window.
 - Keep the stream registry regional and cached per host so a registry outage affects policy changes, not the append path.
 - A per-host disk failure loses at most 24 h of unshipped local data
 - The upload SLO bounds that to minutes for shipped streams.
- Weakest assumption
 - Healthy storage sustains about 20 flushes/s of 100 KB under 50 ms
 - A shared or virtualized disk under neighbor load can stall for seconds and turn the whole queue into the loss window.
 - Mitigation: measure flush latency p99 on the target host class, bound queue age, and expose disk runway before promising any durable-mode latency.
- Over-engineering warning
 - Do not promise a 50 ms maximum loss window from a periodic flush setting alone.
 - Do not build a replicated distributed log for a host-local daemon whose central copy already lives in object storage.

Level signal — IC3 baseline vs IC4 stretch

Phase	IC3 must land	IC4 adds	Separating signal
Requirements	Three acknowledgment modes; ingest versus event time; retention budget	Names the audit stream as a separate product with its own contract	Separates promises instead of tuning one flush knob
Estimates	2 MB/s, 64 MB segment, 20 flushes/s, 27 h disk runway	Lets flush count and disk runway pick group commit and central upload	A number changes a decision
Architecture	Bounded queue, single writer, manifest, uploader, cached registry config	States the cost of each split and the two durability states it creates	Every component has a mechanism and a price
Deep dive	Group commit, durable prefix, atomic manifest rotation, tail recovery	fsync versus directory sync versus fdatasync; unacknowledged durable frame is duplication, not loss	Reasons at the filesystem boundary, not the language API
Operations	Failure table, queue age, disk runway, reader pins	Registry outage as static stability; format-versioned rollback; per-stream shedding	Volunteers degradation and rollout before being asked
Wrap	Recap and tradeoff	Names the disk-stall assumption first	Identifies the flaw before the interviewer does

Interview pushback and validation

- Q1: Is queue acceptance durable?
 - **IC3 baseline**
 - No
 - It only reserves volatile memory.
 - **IC4 stretch**
 - Name separate local and replicated acknowledgment contracts.
- Q2: The disk stalls for 30 seconds
 - How much can be lost?
 - **IC3 baseline**
 - Queued and unflushed records can exceed the nominal flush interval.
 - **IC4 stretch**
 - Bound admission and make durability futures fail or block within their deadlines.
- Q3: Why does a single-predecessor time search miss records?
 - **IC3 baseline**
 - Event-time segment ranges can overlap because events arrive late.
 - **IC4 stretch**
 - Interval indexing or bounded late-event partitions change the query architecture.
- Q4: The caller requires audit persistence before completing a payment.
 - **IC3 baseline**
 - Use a durable mode and clarify the atomicity boundary with the business write.
 - **IC4 stretch**
 - Use a transactional audit intent/outbox where the business database can support it.
- Q5: Can retention delete a file while a reader uses it?
 - **IC3 baseline**
 - Use reader pins or a defined snapshot/expiry contract.
 - **IC4 stretch**
 - Keep reclamation behavior portable across local and object storage implementations.
- Q6: Is compression free on immutable data?
 - **IC3 baseline**

- Immutability removes write races, not CPU/I/O cost.
- **IC4 stretch**
 - Block compression changes seekability and indexing design.
- Q7 (premise reversal): Why not send every record straight to the central service and skip the local file?
 - **IC3 baseline**
 - The hot path takes a network dependency
 - A central outage becomes a local write failure or an unbounded client buffer.
 - **IC4 stretch**
 - Do it only for the audit stream through an outbox in the business database
 - Keep diagnostics local-first with bounded central staleness.
- Q8: The stream registry is down
 - Can hosts still log?
 - **IC3 baseline**
 - Yes, on cached last-known-good config
 - Only new streams and policy changes wait.
 - **IC4 stretch**
 - Bound the staleness and reconcile quota and retention drift on recovery.
- Validation scenarios
 - Partial frame write.
 - Crash between flush and acknowledgment.
 - Crash during manifest replacement.
 - Disk-full group commit.
 - Late event overlapping multiple earlier segments.
 - Retention while a reader pins a segment.
 - Producer batch replay with the same (epoch, seq).
 - Registry unavailable during a retention change.

Recall card

- "Separate memory acceptance, local durability, and remote durability."
- "One writer owns framed append and the durable prefix."
- "Crash recovery validates the tail; retention respects readers and upload policy."
- "Flush interval is a target, not a proof of maximum data loss."
- "Hosts log on cached config; a registry outage stays in the control plane."
- Wrap-up script
 - "The design is a bounded queue, one writer per active segment, checksummed frames, and a manifest that readers pin and the uploader ships."
 - "The key invariant is that the published durable offset never exceeds a successfully flushed prefix, so acknowledgment and durability cannot disagree."
 - "The main tradeoff is group commit: it buys durable throughput at the price of a queue that becomes the loss window under a disk stall."
 - "My weakest assumption is healthy flush latency; I would measure it on the real host class before promising any durable-mode latency."

References

- [Ranked question and report provenance.](#)
- [OCI role framing.](#)
- [Detailed API and schema companion.](#)
- Reference check: 2026-09-09.
 - Official documentation supports the mechanisms cited next to each link.

- Capacity values and architecture choices remain proposed interview assumptions.
- [Linux fsync manual](#): file and directory durability
 - `fdatasync` metadata caveat.
- [Kafka storage design](#): segmented log, sparse offset index, and durability tradeoffs.
- [Object-storage supporting design](#): retained background reference for the upload target.